Engrain: Constrained Decoding with the Parser Resident on the GPU

Anonymous Authors[1]

Abstract

Constrained decoding restricts a language model to a formal language by masking the logits at every step, and every deployed implementation computes that mask on the host. This is not an efficiency detail: a CUDA graph is a recorded sequence of device work, so a component that requires the host mid-step cannot be inside the graph a serving engine replays for its decode loop, cannot hand an allowed set to a fused sampler, and cannot follow a speculative draft without a synchronization per position. We present engrain, a constrained-decoding engine whose LALR(1) parser state — lexer state, parser stack, and conflict forks — lives in GPU memory and advances in device kernels, so that the grammar is part of the decode step rather than an input to it. The design contributes an exact device encoding of the (lexer state × parser stack) configuration space, a mask fill that is a single launch for an arbitrarily heterogeneous batch, and a cross-step memo that answers a third of all steps without touching the tables. On an A100 with a 151,669-token vocabulary, a full constrained step (mask and parser advance) costs 103–327 µs at batch 512 against XGrammar's 2,041–6,811 µs, adds 90 µs to an overlapped decode step where a host matcher adds 398, holds its p99 flat when the host loses cores to other work while the host matcher's p99 rises 7.8×, and is 1.35–4.22× cheaper under speculative decoding. Masks are bit-exact against a reference matcher over 7,198 verified rows. We also report where residency is paid for: at batch 1 we are 0.24–0.90×, our compiler is 7× slower, our resident tables are 27× larger at the median, and our lowering still accepts documents its schema rejects 19.6% of the time against XGrammar's 5.4% — an audit that located and fixed a silent required-dropping defect along the way.

1 Introduction

Structured generation — forcing a language model's output to be valid JSON, to match a schema, or to parse under a grammar — is now a standard serving feature. It is implemented by constrained decoding: before each token is sampled, the engine computes a bitmask over the vocabulary marking which tokens can extend the output without leaving the language, and sets the rest to $-\infty$.

Every widely deployed implementation — XGrammar (Dong et al., 2025), llguidance, Outlines (Willard & Louf, 2023), SynCode (Ugare et al., 2025) — computes that mask with a CPU automaton and copies it to the device. The choice looks reasonable: the mask is small, the automaton step is a few microseconds, and the copy overlaps the forward pass. Measured as a fraction of a decode step, it is 4.6% at batch 512 on a 0.6B model, and an engine that made it twice as fast would be 2.3% faster end to end. Treated as a latency problem, constrained decoding is nearly finished.

This paper argues that latency is the wrong frame, and that the interesting property is where the state lives.

Modern serving engines record their decode step as a CUDA graph and replay it, because at batch 512 the per-kernel launch cost otherwise dominates. A CUDA graph is a fixed, recorded sequence of device work. Anything the host has to produce in the middle of a step cannot be recorded into it — attempting to capture a host-side mask fill yields a graph that contains none of the work, and replaying it reproduces whatever the host buffer happened to hold last. Consequently, every deployed constrained decoder sits outside the graph and hands a buffer in.

Three things follow, and none of them is a speed argument:

- The decode loop cannot be captured whole. A host-dependent component cannot join a fully graph-resident or megakernel decode loop at all.

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. AUTHORERR: Missing \mlsyscorrespondingauthor.

Preliminary work. Under review by the Machine Learning and Systems (MLSys) Conference. Do not distribute.

- The sampler cannot be fused with the constraint. If the allowed set is a device object, a sampler can read the few hundred candidates the grammar permits instead of all 151,669 logits. If it is produced on the host, the only interface is a vocabulary-wide mask.
- Speculative decoding needs a host round trip per draft position, or a specialized batch API, because the parser cannot be advanced and rolled back inside the same recorded step.

On each of these the comparison is not "we are faster" but "the other design cannot participate". That is the claim this paper makes, and the engineering it requires is the paper's substance: moving an LR parser onto a GPU is not a port.

Why an LR parser, and why that is hard. A grammar-constrained decoder must answer, at every step, "which of the model's tokens may come next". Model tokens are byte strings from a BPE vocabulary; they straddle lexeme boundaries, so a token may end in the middle of a number or a string. The parser configuration must therefore carry a lexer state as well as a parser stack. Worse, an LR parser's next state after a reduction depends on the stack below the popped states, so next[state, token] is not a function of the state alone and no flat per-state table is exact. Prior GPU work avoids this by restricting the grammar class: an acyclic schema DFA, a bounded stack depth, or a conservative stack abstraction. Restricting the class is precisely what makes those systems unable to serve the grammars people write.

Contributions.

1. An exact device representation of the (lexer state $\times$ parser stack) configuration space for LALR(1) grammars, including runtime forking on reduce/reduce conflicts, so that no bound truncates the accepted language (§3.1).
2. A mask fill that is a single kernel launch for a batch of arbitrarily many distinct grammars at arbitrarily different document positions, built as a probe that discovers per-sequence work and a sweep that redistributes it — and evidence that the obvious single-kernel fusion is 16.4$\times$ slower on the batch shape serving actually produces (§3.2).
3. A cross-step memo, keyed on the configuration set, that answers 35.6% of real decoding steps without reading the tables (§3.4).
4. Fused constrained sampling: the allowed set is compacted on device and consumed directly by the sampler (§3.5), together with an honest account of why the win does not survive a ragged batch.
5. A differential correctness methodology, and 7,198 verified mask rows across four independent verifications (§6.7).
6. An evaluation that reports the costs of residency — compile time, resident memory, small-batch latency — as first-class results rather than limitations (§6.8).

2 Background and Motivation

2.1 Constrained decoding

Given a context-free grammar G over a terminal alphabet and a model vocabulary V of byte strings, a constrained decoder maintains, per sequence, an automaton state and computes at each step the set $A \subseteq V$ of tokens t such that the output so far concatenated with t is a prefix of some string in $L(G)$. The mask is the indicator of A.

XGrammar (Dong et al., 2025) — the most widely deployed implementation, and our baseline throughout — compiles the grammar to a pushdown automaton, precomputes for each automaton state an "adaptive token mask" partitioned into accepted and rejected sets, and at each step walks the per-sequence stack on the CPU to fill a bitmask, which is copied to the device and applied by a small kernel. Its compiler cache is kilobyte-scale and its compile time is milliseconds, both of which are properties we do not match and report honestly in §6.8.

2.2 The graph boundary

At batch 512, a decode step of a small model issues hundreds of kernels whose individual launch overheads exceed their execution times. Serving engines therefore record the step once as a CUDA graph and replay it. Capture is stream capture: the operations enqueued on a stream between cudaStreamBeginCapture and cudaStreamEndCapture become graph nodes. Host code executes normally during capture and contributes nothing.

This makes the boundary sharp rather than gradual. A constrained decoder that computes its mask on the host is not "slower inside a graph"; it is absent from the graph, and the replayed step silently uses a stale buffer. The only sound arrangement is the one deployed today: keep the fill outside, synchronize, hand

Table 1. Allowed-set width over real decoding steps. The median step allows a few hundred tokens regardless of vocabulary size, but a third of steps allow more than 8,192 — the distribution is bimodal, not concentrated. Source: results/rigorous-summary.json.

tokenizer	$\lvert V\rvert$	steps	median $\lvert A\rvert$	$\lvert A\rvert > 8\text{k}$
Llama 3	128,256	55,406	396	32.9%
Qwen 3.6	248,077	66,557	378	32.0%
Gemma 4	262,144	69,313	107	32.3%

in a buffer. Every property in §1 follows from that arrangement, not from the fill's cost.

2.3 What the workload actually looks like

Two properties of real constrained decoding shape the design, and both are measured over model-generated documents rather than assumed. We generate 533 JSON values with Llama-3-8B-Instruct under an XGrammar constraint (50 schemas from each of the 11 JSONSchemaBench (Geng et al., 2025) configs, temperature 0.8, top-p 0.95, mean 125 tokens) and replay the text through each tokenizer's grammar, which is faithful because the state a matcher reaches is determined by the bytes consumed.

Table 1 shows the first property: width is set by the schema, not the vocabulary. A structural position (after a comma, inside an object) allows a few hundred tokens whatever the vocabulary size, so the ratio between an $O(|V|)$ mask and an $O(|A|)$ allowed set grows as vocabularies grow. The second property is that the distribution is bimodal: string bodies allow nearly everything, and a third of all steps are in one. Any design that assumes narrow rows — including our own fused sampler — meets that third of steps, and §3.5 reports what happens when it does.

The third property is about batch shape, and it caused the largest single regression in this project. A serving batch is skewed by construction: continuous batching admits requests at different times, so every sequence sits at its own position in its own document, and the amount of parser work each needs differs by an order of magnitude. Benchmarks that place every sequence in the same parse state measure the one arrangement that flatters a design which deduplicates. All measurements in this paper seed each sequence at a random position in its document.

3 Design

3.1 The automaton, and where it ends

A configuration is a pair (lexer DFA state, parser stack). The compiler produces, from a JSON Schema, EBNF or regex front end: an LALR(1) ACTION/GOTO table over grammar terminals; a byte-level lexer DFA whose accepting states name terminals; and a partition of the model vocabulary into token groups.

Token groups. The bridge between BPE tokens and grammar terminals is the expensive object. For each lexer state ℓ and each token t, feeding t's bytes from ℓ either fails, or ends in some lexer state ℓ' having emitted a (possibly empty) sequence of terminals. Two tokens that do the same thing from every lexer state are interchangeable for masking purposes, so the vocabulary is quotiented by that equivalence. Real vocabularies collapse hard: 151,669 Qwen3 tokens become a few hundred groups per grammar. The mask fill then works over groups, and expands to tokens only at the end.

Stacks, exactly. The classical obstacle is that a reduction pops k states and the following GOTO depends on what is exposed underneath, so a flat next[state, token] table is not exact. Bounding the stack depth and enumerating reachable stacks — the approach a prototype of this system took — both explodes (compile timed out beyond depth 6 at a 4,096-token vocabulary) and silently makes the accepted language a strict subset of the grammar when the bound is exceeded. That is a correctness cliff, not a performance one.

Engrain keeps the stack. A sequence's device state is a bounded-capacity array of configurations, each holding a lexer state and an explicit parser stack, and the advance kernel performs real reductions with real GOTO lookups. What is bounded is the number of simultaneous configurations, not the depth of any stack, and exceeding that bound is reported as a typed refusal at compile time rather than silently narrowing the language.

Conflicts, forked at runtime. LALR(1) construction on lowered JSON Schema produces reduce/reduce conflicts — oneOf in 86% of the conflicted grammars. Rather than refuse those grammars (which would cost a fifth of the corpus) or promote the whole system to full GLR, engrain forks the conflicted configuration at runtime and carries both, pruning any fork that dies on the next terminal. This is GLR-lite: unbounded in principle, bounded by the configuration capacity in practice, and the capacity is checked rather than assumed.

3.2 The fill: a probe and a sweep

The mask fill is the step's device-side work. Its input is, per sequence, a set of configurations; its output is a bitmask row over the vocabulary. Conceptually it is: for each sequence, for each configuration, for each token group reachable from that configuration's lexer state, if the group's terminal is accepted by ACTION in the configuration's parser state, OR the group's token bits into the row.

The iteration space is therefore ragged in three dimensions at once — sequences have different numbers of configurations, configurations have different numbers of groups, and groups have different numbers of tokens. Our first CUDA implementation fused the whole thing into one kernel with one block per sequence. On a balanced batch it was 2.3–4.5× faster than the Triton implementation it replaced. On a skewed batch — the shape serving actually produces — it was 16.4× slower (9,883 µs against 604 µs), because one block per sequence makes the fill cost the widest sequence rather than the total work. Measuring a fused kernel on a balanced batch is not measuring it.

The fix is two kernels, and the reason it must be two is a hazard rather than a preference:

- Probe (one block per sequence) clears the sequence's row and records how many (configuration, group) items it will contribute.
- Sweep (grid (batch, chunks)) walks a flattened (configuration, group) iteration space, so a sequence with 64 configurations of one group each is spread over blocks rather than serialized into one.

They cannot be merged: the probe clears rows that the sweep ORs into, and a block clearing a row while another accumulates into it loses bits. The chunk count is bounded by a scratch budget rather than by the batch — a thread owns two replay windows, so the buffer is batch × chunks × threads, which is 268 MB at batch 512 with 8 chunks. A 64 MiB budget gives 32 chunks at batch 32 and 2 at batch 512.

After the split the skewed step fell from 9,883 µs to 641 µs, a 15.4× improvement, and the remaining gap to our own Triton reference narrowed from 1.16–3.29× to 1.16–1.31×.

3.3 The advance

After a token is sampled, each sequence's configurations must be advanced by it. This is the half that cannot be hidden behind the forward pass, because it follows the sampled token. Engrain fuses the whole advance — feeding the token's bytes through the lexer, running reductions and GOTOs, forking on conflicts, compacting dead configurations, and writing the rollback history entry — into a single kernel. The rollback path went from 16–20 graph nodes to 7 by folding the history write inline, removing a counting kernel and a grid $= (1,)$ prefix sum.

3.4 Cross-step memoisation

Sequences repeat states. Within one request, consecutive steps often reach a configuration set that has been seen before; across a batch under one grammar, many sequences share one. Engrain hashes the configuration set on device — one warp per configuration, folding stacks in shared memory — and looks it up in a device-resident memo of previously computed rows. Over a real walk the memo answers 35.6% of steps without reading a table.

The hash itself was a bottleneck until it was parallelized: one warp performing 64 sequential stack reductions cost 65.8 µs; one warp per configuration costs 16.3 µs, half of the Triton reference's 32.8.

3.5 Fused constrained sampling

Because the allowed set is a device object, the sampler can read it directly. A compact kernel emits, per row, a sorted list of allowed token ids using a block-wide scan. The sampler then gathers a few hundred logits instead of reading all 151,669.

The honest accounting of this idea is in §??, and it is not the one this paper originally claimed. Compaction wins by 7.7× on a sparse row and loses by 0.95× on a dense one, so a fixed policy is a net loss on the real bimodal distribution; a per-step choice made on device (emit whichever of the allowed set and its complement is shorter) recovers 1.79×; and on a genuinely ragged batch — half the rows dense, which is the measured steady state — the advantage falls to 1.05×. The reason is not in the parser: it is that XGrammar's mask-apply takes its row indices as a host List[int], and a sampler's output shape is its row count, so any subset scheme needs a host synchronization that the residency claim exists to avoid.

3.6 Arena and paging

All grammars the engine has seen live in one device arena, and a sequence carries the index of the grammar it is under, which is what makes a batch of distinct grammars a single launch. Under continuous batching the arena must reclaim memory, and the hard part is doing so without moving anything: compaction renumbers survivors, and every recorded CUDA graph would have to be re-recorded, giving back exactly what residency bought. Engrain therefore gives each array a coalescing free list and recycles identifiers rather than renumbering them, pinning any grammar a live sequence is running under and evicting the rest least-recently-used. Across 400 admissions of 40 schemas at a 32 MB budget, 399 evictions cost zero graph re-records.

4 Implementation

The compiler is 7 Rust crates ($\approx$20k lines): front ends (JSON Schema, EBNF, regex), lexer construction and vocabulary grouping, LALR(1) table construction, artifact emission, and a reference matcher. The device runtime is CUDA compiled to a fatbin for sm_80–100 plus forward-compatible PTX, embedded in the extension so that no driver JIT happens at load; the Python layer is $\approx$5k lines over PyO3 bindings.

The system keeps two complete backends: the CUDA kernels, and a Triton implementation of the same algorithms that is retained as an executable reference. A third differential backend runs both on every call and compares, which is how kernel-level divergence is caught. All results in this paper are from the CUDA backend unless stated.

5 Experimental Design

A systems paper's claims are only as good as the angles from which someone tried to break them. We state each claim with the measurement that would falsify it, and report all of them — including the two that came out against us and the one that found a bug in our own compiler.

Two design rules apply throughout. Both halves are charged: a step is a mask fill and a parser advance, and reporting only the fill flatters whichever engine has the cheaper one. Batches are skewed: every sequence is seeded at a random position in its own document, because a balanced batch is the single arrangement that flatters a design which deduplicates, and measuring an ablation on one would understate every mechanism at once.

Table 2. The claims, and the measurement designed to falsify each. Every row is reported in §6, including those whose outcome is negative.

claim	what would falsify it
the fill can live in the decode graph	a graph recorded on one grammar assignment replaying wrongly on another
residency is worth having	XGrammar's fill overlapping the forward pass so completely that its added cost equals ours
the host is a liability	the host matcher's tail staying flat as CPU cores are taken away
we are cheaper per step	llguidance or a better-tuned XGrammar closing the gap
each mechanism earns its place	removing the memo, or capture, changing nothing
heterogeneity is affordable	cost growing with the number of distinct schemas in the batch
masks are exact	a device row differing from the reference matcher
the grammar is the schema	documents the grammar accepts that the schema rejects

6 Evaluation

6.1 Setup

All measurements are on one NVIDIA A100 80GB PCIe with Qwen3-0.6B's 151,669-token vocabulary unless stated, and use three JSONSchemaBench schemas of increasing difficulty (schema 0, 1, 2). Both engines are charged for both halves of a step — the mask fill and the parser advance — because reporting only the fill flatters whichever engine has the cheaper one. XGrammar's thread count is swept over $\{1, 2, 4, 8, 16, \mathrm{auto}\}$ and its best is reported; its device work is CUDA-graphed, as ours is. Every sequence is seeded at a random position in its document. We report medians of 100–200 repeats after warm-up.

6.2 Step latency

Table 3 is the paper's central performance result and also its clearest limitation. At batch 1 we lose on all three schemas; the crossover is between batch 8 and 32 depending on schema; above it the gap widens monotonically to 12.8–20.8× at batch 512.

Two readings matter. First, the shape is structural rather than incidental: XGrammar's cost is per sequence and scales with the batch, ours is a fixed number of launches whose work scales with distinct parse states, so the gap is a property of the two designs and not of tuning. Second, the small-batch loss is also structural: a step sweeps every live configuration's groups whatever the batch, so our floor is 41–49 µs

Table 3. Full constrained step (fill + advance), median µs, three schemas. "Triton" is our own reference implementation of the same algorithms, included so the CUDA work is measured against the engine it replaces and not only against the baseline. Source: engrain_lab.rigor.latency.

batch	schema	engrain	Triton	XGrammar
1	0	49	—	12
	1	46	—	14
	2	41	—	37
16	0	57	—	70
	1	133	—	82
	2	162	—	363
32	0	59	83	130
	1	136	116	150
	2	194	147	659
128	0	63	123	537
	1	139	156	607
	2	257	204	2,073
512	0	103	156	2,041
	1	185	203	2,375
	2	327	247	6,811

Table 4. Cost of a batch drawn from n distinct schemas, median µs, and the cost of mixing itself: the ratio of the mixed batch to the mean of the same schemas run alone at the same batch size. Source: probe_mixed_cost, 2026-07-31.

batch	n	engrain	XGrammar	ratio	mix costs us
32	1	82	166	2.01×	0.99×
	2	388	181	0.47×	5.00×
	4	401	398	0.99×	5.38×
	8	442	1,991	4.51×	5.50×
	16	394	1,086	2.76×	5.19×
128	1	87	529	6.09×	1.00×
	2	390	663	1.70×	4.74×
	4	406	1,595	3.92×	5.11×
	8	1,614	9,594	5.94×	18.60×
	16	453	7,015	15.50×	5.54×
512	1	114	1,930	16.89×	0.99×
	2	419	2,456	5.86×	3.79×
	4	430	6,604	15.37×	4.02×
	8	4,025	35,781	8.89×	33.17×
	16	4,052	44,963	11.10×	35.98×

at batch 1 while XGrammar's per-sequence walk costs 12–37 µs. We do not claim a small-batch win, and a serving deployment that runs at batch 1 should not use this system.

Table 3 also shows that our CUDA implementation still trails our own Triton reference on the hardest schema (1.24–1.32×). The cause is per-item throughput in the sweep, not load imbalance, and we report it because an implementation that loses to its own reference has not finished.

6.3 Heterogeneous batches

The case the design exists for is a batch of different grammars, which is what continuous batching produces and what should hurt us: our fill deduplicates rows that share a grammar, a lexer state and a stack, and a mixture has fewer such rows to share, while our buffers are sized by maxima over the whole pool. XGrammar's per-sequence host call is schema-agnostic, so mixing costs it nothing structurally.

Table 4 reports both the comparison and the control. Three findings, one of them against us:

Heterogeneity is a step, not a slope. At batch 32, one grammar costs 82 µs and two cost 388; but sixteen also cost 394. The 5× is paid once, when the pool stops being homogeneous, and then does not grow with the mixture. This is the property that matters for serving, where the mixture is never one.

We lose at two schemas and small batch. 0.47× at batch 32 with two schemas is the worst cell in the table, and it is the direct consequence of the step above: we pay the full heterogeneity cost while XGrammar still has only 64 cheap host calls to make.

The absolute cost at 8–16 schemas and batch 512 is bad. 4 ms is 33–36× what the same schemas cost run alone, against 0.82–1.04× for XGrammar. We still win the comparison 8.9–11.1× because their absolute cost is 36–45 ms, but the honest statement is that pool-wide ceilings — the replay window, the readings a group may have — are sized by the largest member and every sequence pays for it. This is the clearest target for future work.

6.4 Speculative decoding

A draft of k tokens must be checked against the grammar and the accepted prefix kept. This is the case where a host-side matcher stops being merely costly. XGrammar offers traverse_draft_tree, which walks the whole draft tree in one call and is close to flat in k; comparing against its per-position path instead — as an earlier version of this work did — overstates the gap by roughly 46× and we do not do so.

We win at every k measured (Table 5), but the trend is against us: our cost grows with k (278/474/730 at batch 512) because each draft position pays a fill and an advance, while theirs is roughly flat. Extrapolating, the advantage closes near $k \approx 16$. The structural point

Table 5. Speculative step, median µs, against traverse_draft_tree. Every position pays a fill and an advance on our side, and the rollback is charged. Source: probe_speculative, 2026-07-31.

batch	k	engrain	XGrammar	ratio
128	1	102	260	2.56×
	4	219	562	2.57×
	8	361	485	1.35×
512	1	278	1,072	3.85×
	4	474	1,999	4.22×
	8	730	1,980	2.71×

Table 6. Cost added to a real decode step when the constraint is overlapped with the forward pass. The comparison the objection asks for. Source: engrain_lab.rigor.overlap.

batch	forward	engrain adds	XGrammar adds	ratio
32	6,559 µs	44 µs	46 µs	1.05×
128	9,394 µs	60 µs	112 µs	1.87×
512	22,090 µs	90 µs	398 µs	4.42×

Table 7. Mask fill at batch 128 while n cores are kept busy beside it. Source: engrain_lab.rigor.serving.

busy	engrain		XGrammar	
cores	p50	p99	p50	p99
0	27.8	51.0	406.5	449.6
8	27.5	66.5	433.2	464.2
16	27.8	59.9	421.5	448.3
24	27.9	56.7	444.7	3,495.3

Table 8. Ablation on a three-schema batch with sequences at random document positions. "memo" compares the full memo against a single-slot one; at batch 1 that is degenerate, since one slot always serves one sequence. Source: exp_ablation.

batch	full	memo worth	capture worth	memo hit
1	49.8 µs	1.00× †	2.56×	100.0%
8	59.6 µs	6.15×	2.34×	100.0%
32	408.6 µs	1.13×	1.04×	93.8%
128	448.7 µs	1.16×	1.04×	90.6%
512	3,980.4 µs	1.00×	1.00×	86.1%

survives — our walk is one recorded graph replayed once, with rollback on device, and needs no synchronization per position — but the arithmetic advantage is finite and we say so.

6.5 The two strongest counter-arguments

Per-step numbers invite two objections, and both are stronger than any ratio we can quote. We measured them rather than argued with them.

"XGrammar overlaps its fill with the forward pass, so residency buys nothing." This is the objection that once forced this project to withdraw a claim. We measure a real decode forward pass, each engine alone, and the two together on one stream, and charge each engine only the time it adds to the step (Table 6). At batch 32 the objection holds: we add 44 µs and XGrammar adds 46, which is parity. It stops holding as the batch grows, because what overlaps is bounded by the forward pass and XGrammar's cost is not: at batch 512 it adds 398 µs to a 22 ms step against our 90.

"The host is idle anyway." A host-side matcher is cheap when it has a core to run on. Serving hosts do not offer that guarantee: the same CPUs run detokenization, scheduling, request parsing and the API server. We sweep the number of busy cores next to the fill (Table 7). Our p50 and p99 are flat — 27.8 to 27.9 µs and 51.0 to 56.7 — because the work is on the device. XGrammar's median degrades mildly (1.09×) but its p99 goes from 449.6 to 3,495.3 µs, a 7.8× tail blow-up. Constrained decoding is a tail-latency business, and this is the clearest measurement in the paper of what residency is actually for.

6.6 Ablation: what each mechanism is worth

Table 8 removes one mechanism at a time. Both knobs are set before the step is captured, which matters more than it sounds: our first attempt set them afterwards and reported that the memo was worth 1.00× while hitting 90% of steps — the captured graph had the old scalar arguments baked in, so nothing had actually been disabled. An ablation of a graph-resident system has to be recorded, not configured.

The two mechanisms pay in the same place and for the same reason: at small batch the step is dispatch-bound and mostly redundant, so capture is worth 2.3–2.6× and the memo up to 6.2×; at batch 512 both are 1.00× because the step is work-bound and the work is genuinely different per sequence. We include the negative half of this result because a mechanism that helps only at batch 8 should be described that way.

The third ablation is the fill split (§3.2), and it is the one that justifies the paper's implementation lesson: on a balanced batch the single fused kernel is fine, and on a skewed one it costs 9,883 µs against 691 µs. The regression is invisible to any benchmark that does not

Table 9. Constrained sampling at batch 512, measured in place (no restore of a 310 MB logits tensor that a decode loop never pays). "Shortlist" emits whichever of the allowed set and its complement is shorter, decided on device. Source: session measurements, 2026-07-30.

regime	vs. mask-then-sample	note
all rows sparse	7.73×	upper bound
all rows dense	0.95×	compaction frees
distribution-weighted	1.79×	shortlist, per-step choice
ragged batch, 50% dense	1.05×	measured steady state
ragged, sync-free scheme	0.86×	fixed shape, no host sync

Table 10. Random-walk generation under each engine's mask on the 194 schemas both engines compile, same seed, same 400-byte budget. "Before" and "after" are the compiler defect described below. Source: exp_soundness_split.

	completion	valid | complete	aggregate
engine, before	52.5%	70.0%	36.7%
engine, after	50.5%	80.4%	40.6%
XGrammar	63.7%	94.6%	60.2%

seed sequences apart.

Table 9 is the result we most want to be different. Fused sampling is the cleanest consequence of residency — constraining makes sampling cheaper, which is impossible if the constraint is a host product — and on a homogeneous sparse batch it delivers 7.7×. But the measured steady state of a continuously batched workload is about half dense rows (§2), and there the advantage is 1.05×: parity.

We report the cause precisely because it is not in our engine. Two interfaces prevent the ragged case from paying: XGrammar's apply_token_bitmask_inplace takes its row indices as a host List[int], and a sampler's output shape is its row count, so selecting a subset of rows requires knowing that count on the host. What is needed is a sampler that accepts a device-side row count. Until sampling APIs offer one, the fused-sampling benefit is available in principle and 5% in practice on a mixed batch.

6.7 Correctness, in two layers

Constrained decoding admits no approximation: one wrong bit either leaks an illegal token or removes a legal one, so learned or lossy mask representations are out of scope by construction. But "correct" names two different properties, and conflating them is how this literature reports numbers that cannot be acted on:

runtime exactness the device mask equals what the compiled grammar allows;

lowering fidelity the compiled grammar equals the schema.

The first is what residency depends on and we verify it exhaustively. The second is where every grammar-based system loses ground, and it is where we found — and fixed — a real defect in our own compiler.

Runtime exactness. Every mask row the device produces is compared bit-for-bit against an independent CPU reference matcher built from the same tables. Four verifications run over the real corpus: corpus (619 rows over model-generated documents), walk (4,800 rows over random walks, which reach states real documents do not), conflicted (687 rows restricted to grammars with reduce/reduce conflicts, i.e. where forking is live), and mixed (1,092 rows over batches of several grammars, including a CUDA graph recorded on one assignment of grammars to sequences and replayed on a different one). All 7,198 rows match, with zero failures. The graph-replay check is the one the residency claim rests on: a recorded step survives the batch composition changing underneath it, which is what continuous batching does every step. In addition, 99 unit tests run under all three backends, and the differential backend runs the CUDA and Triton implementations on every call and compares.

Lowering fidelity, and a metric that was measuring the wrong thing. The aggregate we had been reporting — "56.0% of documents generated under our mask validate, against XGrammar's 77.6%" — turns out to conflate two unrelated quantities, because a random walk that runs out of byte budget produces truncated JSON and was counted as a failure. Decomposing it (Table 10) separates completion (did the walk finish a document) from over-acceptance (given a complete document, does the schema accept it). Only the second is a property of the engine.

Attributing the gap to a keyword found a bug. Recording which JSON Schema keyword rejected each over-accepted document showed the failure was not diffuse: 219 of 243 over-acceptances were the single keyword required, against 8 for XGrammar. That concentration is what made the cause findable. It was not the precision level — forcing the most faithful lowering moved 181 of 194 schemas from Shadowed to Exact, cost 4× compile time and 2× tables, and left validity unchanged at 70.0%, which ruled out the hypothesis we started with.

The cause was in the order-free object construction. To let properties arrive in any order while still enforcing required, the compiler enumerates subsets of the required set and carries the subset in the parser state, one rule per subset. For the full subset — every required property seen — the rule that picks the next property has no alternatives when the object also closes its property names (additionalProperties: false). The code treated “no property may follow” as a construction failure and fell back to an object that does not enforce required at all. The condition is not a failure; it means the object must now close. Minimally: an object with one declared property, that property required, and additionalProperties: false accepted {}. With two properties it did not, which is why no unit test caught it.

Fixing it removes 78 of the 243 over-acceptances, eliminates the oneOf failures entirely, and moves validity from 70.0% to 80.4% (Table 10). We report the before and after because the methodology — attribute, minimize, fix, re-measure — is the contribution here, not the number.

What remains, and the honesty defect behind it. 141 required failures remain, and they have a different cause: the subset enumeration costs 2^n rules, so it is bounded at 4 required properties when the property names are open and 6 when they are closed. Past that the object widens. That is a defensible budget — the alternative, fixing the property order as XGrammar does, rejects valid permutations, which this system's contract forbids — but it was being applied silently: 33 of the 34 corpus schemas whose required was dropped reported no approximation at all, because the approximation list was derived from the precision level rather than from what the lowering did. The engine now reports it in every case, which takes the silent count to zero. We consider a constrained decoder that relaxes a schema without saying so to be the more serious of the two defects, and we suspect the distinction between coverage, exactness, and declared exactness is under-reported across this literature.

6.8 The costs of residency

Residency is not free, and the honest accounting is the following three results. All are measured over 528 distinct JSONSchemaBench schemas.

Compile time (Table 11) is 7× XGrammar's at the median and worse in the tail. It is, however, linear in the number of lexer states (slope 1.05, $r = 0.94$): there is no algorithmic blow-up, only the inherent $O(|V| \times$ lexer states) cost of computing what every token does from every lexer state. In a serving context, schemas arrive with requests, so this is a stated failure and not a footnote.

Table 11. What residency costs. Compile time and table memory are measured over 528 JSONSchemaBench schemas; XGrammar's figures are for the same schemas. Source: engrain_lab.rigor.cost, 2026-07-30.

	engrain	XGrammar	ratio
compile p50	112 ms	16 ms	7.0× worse
compile p90	1.38 s	—	
compile p99	4.63 s	0.6 s	7.7× worse
compile max	12.5 s	—	
tables p50	1.41 MiB	52 KiB	27× worse
tables p99	37.5 MiB	—	700× worse
tables max	97.6 MiB	—	
schemas compiled	502/528 (95.1%)	≈98%	−3 pts

Memory. 1.41 MiB at the median against a 52 KiB compiler cache. We looked for structural sharing and did not find much: whole-array sharing across schemas gives 1.0× (nothing is byte-equal), group sets are already interned, and interning reading lists saves 12%. The one lever with the right shape is building a lexer state's reading tables on demand, the first time a parse reaches that state, since a real document visits only 1–4% of the states its grammar can reach. That would address compile time and memory with one mechanism, and it is the main piece of future work.

Coverage. 502 of 528 schemas compile (95.1%). The 26 refusals are typed: 12 the front end cannot lower, 10 exceed the production budget, 4 exceed the lexer budget. Separately — and this is a distinction the constrained decoding literature does not usually draw — 371 of the 502 compiled grammars (73.9%) accept a relaxation of their schema rather than exactly the schema. The relaxations come from the lowering, not the runtime: oneOf becomes a union, and a union is “at least one” where oneOf is “exactly one”. The runtime only ever narrows. We report bit-exactness against our compiled grammar, which is a weaker and more precise claim than bit-exactness against the schema, and we believe every system in this space owes the same distinction.

7 Limitations

We state these as results rather than as caveats, because the design's central claim is structural and survives them, while a reader deciding whether to deploy needs them.

1. Batch 1 is a loss (0.24–0.90×). Our floor is set by sweeping live configurations, not by the batch.

2. Compile is 7× slower and the p99 is seconds. Lazy lexer residency is the plausible fix and is unimplemented.

3. Tables are 27–700× larger than a host matcher's cache.

4. Our lowering over-accepts 19.6% of complete documents, against XGrammar's 5.4%, and 141 of the 153 remaining cases are one keyword, required, dropped where the subset enumeration exceeds its budget.

5. Fused sampling is 1.05× on a ragged batch, not the 7.7× a homogeneous one suggests, and the blocker is in surrounding APIs.

6. Heterogeneous pools of 8–16 schemas cost us 33–36× what those schemas cost alone.

7. The memo and graph capture pay only below batch 32 (Table 8); at batch 512 both are 1.00×.

8. One baseline, one GPU. We compare against XGrammar; llguidance is installed and not yet in the comparison. Every launch shape, memo size and chunk count was fitted on an A100.

9. No end-to-end throughput claim. Grammar work is a few percent of a decode step, and a vLLM A/B at batch 256 ranged from 4,523 to 13,325 tokens per second across runs — too noisy to attribute a difference. We report the per-step result, which is the defensible one.

8 Related Work

Host-side constrained decoding. Outlines (Willard & Louf, 2023) indexes a regular-language DFA against the vocabulary; SynCode (Ugare et al., 2025) and grammar-aligned decoding (Park et al., 2024) extend to context-free grammars; XGrammar (Dong et al., 2025) makes the pushdown automaton practical with adaptive token masks and is the deployed baseline; llguidance and Domino (Beurer-Kellner et al., 2024) attack the same overhead with better host algorithms and speculation. All compute the mask on the CPU, which is the property this paper is about.

GPU-side work. Pre3 (Chen et al., 2025) and PSC (Li et al., 2026) precompute deterministic pushdown structure to make the host step cheaper or to classify parser stacks; Gram2Token (Hua et al., 2026) moves the token-to-terminal bridge toward the device. These reduce the cost of the host step or move part of it; to our knowledge none keeps the parser stack itself on the device across steps, which is what makes graph capture and sampler fusion possible.

Automata and tokenizers. The tokenizer–grammar impedance mismatch is treated by (Koo et al., 2024) and (Park et al., 2025); CRANE (Banerjee et al., 2025) studies the expressiveness cost of constraining. Our token-group quotient is a device-oriented instance of the same idea.

Serving. CUDA graph capture of the decode loop is standard in vLLM and SGLang (Zheng et al., 2024); our contribution is to make the grammar a participant in it.

9 Conclusion

Constrained decoding has been treated as a host-side algorithms problem, and by that measure it is nearly solved. We argued that the interesting question is where the parser's state lives, because a decode step that a serving engine records as a CUDA graph can only contain device work. Engrain keeps an LALR(1) parser — lexer state, stack, and conflict forks — resident on the GPU, which makes the constraint capturable, fusable with the sampler, and advanceable through a speculative draft without a host round trip. It costs 103–327 µs per full step at batch 512 against 2,041–6,811 for the deployed baseline, 1.35–4.22× less under speculation, and is bit-exact over 7,198 verified rows. It also costs 7× more to compile and 27× more memory, loses below batch 16, and delivers only parity on fused sampling once the batch is ragged. We believe the structural claim is worth those prices, and we have tried to report them precisely enough that a reader can disagree.

References

Banerjee, D., Suresh, T., Ugare, S., Misailovic, S., and Singh, G. CRANE: Reasoning with constrained LLM generation. In Proceedings of the 42nd International Conference on Machine Learning, volume 267 of Proceedings of Machine Learning Research, pp. 2836–2857. PMLR, 2025. URL https://proceedings.mlr.press/v267/banerjee25a.html.

Beurer-Kellner, L., Fischer, M., and Vechev, M.

Guiding LLMs the right way: Fast, non-invasive constrained generation. In Proceedings of the 41st International Conference on Machine Learning, volume 235 of Proceedings of Machine Learning Research, pp. 3658–3673. PMLR, 2024. URL https://proceedings.mlr.press/v235/beurer-kellner24a.html.

Chen, J., Bai, S., Wang, Z., Wu, S., Du, C., Yang, H., Gong, R., Liu, S., Wu, F., and Chen, G. Pre^3: Enabling deterministic pushdown automata for faster structured LLM generation. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 11253–11267, 2025. doi: 10.18653/v1/2025.acl-long.551. URL https://aclanthology.org/2025.acl-long.551/.

Dong, Y., Ruan, C. F., Cai, Y., Lai, R., Xu, Z., Zhao, Y., and Chen, T. XGrammar: Flexible and efficient structured generation engine for large language models. Proceedings of Machine Learning and Systems, 7, 2025. URL https://arxiv.org/abs/2411.15100.

Geng, S., Cooper, H., Moskal, M., Jenkins, S., Berman, J., Ranchin, N., West, R., Horvitz, E., and Nori, H. JSONSchemaBench: A rigorous benchmark of structured outputs for language models, 2025. URL https://arxiv.org/abs/2501.10868.

Hua, H., Su, J., Tang, H., Yao, Y., and Zhu, F. Gram2token: Enabling run-time GPU-native grammar-constrained decoding for LLMs. In Proceedings of the 43rd International Conference on Machine Learning, 2026. URL https://icml.cc/virtual/2026/poster/62392. Conference abstract; public implementation was unavailable at survey time.

Koo, T., Liu, F., and He, L. Automata-based constraints for language model decoding. In First Conference on Language Modeling, 2024. URL https://arxiv.org/abs/2407.08103.

Li, Y., Dong, Y., Li, J., and Li, G. Efficient grammar-constrained decoding via parser stack classification. In Proceedings of the 35th ACM SIGSOFT International Symposium on Software Testing and Analysis, 2026. URL https://conf.researchr.org/details/issta-2026/issta-2026-research-papers/81/Efficient-Grammar-Constrained-Decoding-via-Parser-Stack-Classification. Forthcoming; public conference abstract available.

Park, K., Wang, J., Berg-Kirkpatrick, T., Polikarpova, N., and D'Antoni, L. Grammar-aligned decoding. In Advances in Neural Information Processing Systems, 2024. URL https://arxiv.org/abs/2405.21047.

Park, K., Zhou, T., and D'Antoni, L. Flexible and efficient grammar-constrained decoding. In Proceedings of the 42nd International Conference on Machine Learning, volume 267 of Proceedings of Machine Learning Research, pp. 48262–48275. PMLR, 2025. URL https://proceedings.mlr.press/v267/park25l.html.

Ugare, S., Suresh, T., Kang, H., Misailovic, S., and Singh, G. Syncode: LLM generation with grammar augmentation. Transactions on Machine Learning Research, 2025. ISSN 2835-8856. URL https://openreview.net/forum?id=HiUZtgAPoH.

Willard, B. T. and Louf, R. Efficient guided generation for large language models, 2023. URL https://arxiv.org/abs/2307.09702.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. SGLang: Efficient execution of structured language model programs. In Advances in Neural Information Processing Systems, 2024. URL https://arxiv.org/abs/2312.07104.